

Cosmic-Generator

Manual de Usuario - Volumen 2



Cooperativa Central:

Rebeldía Cósmica

Edición Auténtica y Sinérgica – 1 de junio de 2026

Índice general

I	Fundamentos: Física y Complejidad	5
1.	Introducción Teórica a los Motores Físicos de Cosmic-Generator	6
1.1.	La Naturaleza de los Sistemas Complejos	6
1.2.	Arquitecturas Algorítmicas: Separación Conceptual	7
1.2.1.	Modo Neural: Redes Neuronales Productoras de Patrones Composicionales (CPPN)	7
1.2.2.	Modo Caos 3D: Dinámica No Lineal y el Atractor de Lorenz	8
1.3.	Motores de Ecuaciones Diferenciales Parciales (EDPs)	9
1.3.1.	Ecuación de Reacción-Difusión (Modelo de Gray-Scott) . . .	9
1.3.2.	Ecuación de Kuramoto-Sivashinsky (KS)	9
1.3.3.	Ecuación de Gross-Pitaevskii (GPE)	10
1.3.4.	Ecuación de Onda	10
II	Arquitectura Técnica y Audio	11
2.	Arquitectura del Motor Reactivo: Mapeo de Audio y Dinámica Física	12
2.1.	Extracción de Descriptores de Audio	12
2.2.	Mapeo Sincro-Físico: De la Música a la Ecuación	13
2.2.1.	Paso de Tiempo Dinámico (CFL Control)	13
2.2.2.	Modulación del Modelo Gray-Scott (Turing)	14
2.2.3.	Modulación de Ondas y Otras PDEs	14
2.3.	Estabilidad Numérica y Limitadores Anti-Clipping	14
2.4.	Generación Estocástica y Uso de Semillas Aleatorias	15
2.5.	Conclusión del Mapeo	16

<i>ÍNDICE GENERAL</i>	3
III El Nexo Central	17
Sinergia: Física + Programación	18
IV Legado: Manual Volumen 1	20
V Arquitectura del Sistema	21
2.6. Arquitectura del Código y Flujo de Trabajo	22
2.6.1. Punto de Entrada: UI y Configuración (<code>app.py</code>)	22
2.6.2. Orquestación del Sistema (<code>lanzador.py</code>)	22
2.6.3. Patrón Productor-Consumidor en Renderizado (<code>src/render/stable/render_stand</code>)	22
2.6.4. Organización Modular del Código	23
2.7. Módulo: <code>src/core</code> (Motor Numérico y Físico)	23
2.7.1. Archivos Principales y Funcionalidad	24
2.7.2. Cálculos de Dinámica y Cinemática	24
2.8. Módulo: <code>src/audio</code> (Síntesis y Gestión de Sonido)	24
2.8.1. Extracción Acústica (<code>audio_analyzer.py</code>)	24
2.8.2. Alineación de Letras (<code>motor_lyrics.py</code>)	25
2.8.3. Eventos Disparados por el Core	25
2.9. Módulo: <code>src/render</code> (Representación Visual Exclusiva)	25
2.9.1. Estructura del Motor de Renderizado	25
2.9.2. Proceso de Dibujado (Drawing Pipeline)	26
VI El Puente Sensorial (Audio a Física)	27
2.10. Extracción de Parámetros de Audio	28
2.10.1. Energía y Zoom (RMS)	28
2.10.2. Turbulencia y Flujo Espectral (Spectral Flux)	28
2.10.3. Cromestesia y Color (Pitch/Tonalidad)	28
2.11. Frecuencia y Sincronización de Cambio de Parámetros	29
2.11.1. Muestreo Discreto del Audio	29
2.11.2. Time-stepping Loop vs. Render Loop	29
2.11.3. Mutación Dinámica de la Difusión	29
2.12. El Puente Físico-Sonoro: Destruyendo la Caja Negra	30
2.12.1. Extracción de Parámetros de Audio	30
2.12.2. Motor 1: Reacción-Difusión (Gray-Scott / Corrosión)	30
2.12.3. Motor 2: Kuramoto-Sivashinsky (Fuego / Caos Espacial)	31

2.12.4. Motor 3: Gross-Pitaevskii (Cuántico / Superfluido)	32
2.12.5. Motor 4: Onda Clásica	32
2.12.6. Motor 5: Korteweg-de Vries (KdV / Tsunamis, Solitones) . .	33
VII Motores Físicos y Ecuaciones Diferenciales	34
2.13. Fundamentos Físico-Matemáticos de los Motores Dinámicos	35
2.13.1. Motor de Corrosión: Modelo de Reacción-Difusión de Gray-Scott	35
2.13.2. Motor de Fuego: Ecuación de Kuramoto-Sivashinsky	35
2.13.3. Motor Cuántico: Ecuación de Gross-Pitaevskii	36
2.13.4. Motor de Onda: Dinámica Hiperbólica Clásica	36
2.13.5. Motor de Tsunamis: Ecuación de Korteweg-de Vries (KdV) .	36
2.14. Discretización y Simulación Numérica de Sistemas Dinámicos . . .	37
2.14.1. Discretización Espacial y el Operador Laplaciano	37
2.14.2. Condiciones de Frontera Periódicas y Optimizaciones Tenso- rales	38
2.14.3. Integración Temporal: Euler Explícito vs. Runge-Kutta . . .	38
2.15. Fundamentación de los Parámetros Físicos Base	39
2.15.1. Motor Gray-Scott (Reacción-Difusión)	39
2.15.2. Motor Kuramoto-Sivashinsky (KS)	39
2.15.3. Motor Cuántico (Gross-Pitaevskii - GPE)	40
2.15.4. Motor de Onda Clásica	40
VIII Efectos Especiales y Geometría Sagrada (Motores Inéditos)	41
3. Sistema de Partículas: Los Espíritus Procedurales	42
3.1. Código de Emisión (Extracto)	42
4. Gestión Lumínica: Bloom y Flash	43
4.1. Código del Multi-Layer Glow (Extracto)	43
5. Extracción Lirical y Alineación Forzada	44
6. Geometría Sagrada y Caos 3D	45

Parte I

Fundamentos: Física y Complejidad

Capítulo 1

Introducción Teórica a los Motores Físicos de Cosmic-Generator

El universo de *Cosmic-Generator* se fundamenta en la simulación rigurosa y la exploración estética de los sistemas dinámicos, la mecánica de fluidos, la teoría del caos y las redes neuronales generativas. Este documento proporciona una visión profunda y educativa de las matemáticas y la física que operan bajo el capó del sistema, garantizando el rigor necesario para la investigación y el desarrollo continuo.

1.1. La Naturaleza de los Sistemas Complejos

En la física moderna, un sistema complejo se define como un ensamble de múltiples componentes interactuantes cuyo comportamiento macroscópico exhibe propiedades emergentes; es decir, dinámicas que no pueden ser deducidas mediante la simple suma de los comportamientos de sus partes aisladas. La esencia de esta complejidad radica en la **no linealidad**.

A diferencia de los sistemas lineales, donde los efectos son proporcionales a las causas (y aplica el principio de superposición), los sistemas no lineales poseen ciclos de retroalimentación que pueden amplificar exponencialmente pequeñas perturbaciones. Esto nos conduce al concepto de caos determinista: sistemas regidos por ecuaciones deterministas estrictas, pero cuya evolución a largo plazo es intrínsecamente impredecible debido a una dependencia hipersensible a las condiciones iniciales. En *Cosmic-Generator*, aprovechamos estas arquitecturas matemáticas no solo para estudiar el caos, sino para generar texturas visuales, patrones de fluidos y estructuras sonoras en tiempo real.

1.2. Arquitecturas Algorítmicas: Separación Conceptual

Para manejar la inmensa variedad de fenómenos a simular, *Cosmic-Generator* se divide en dos paradigmas generativos fundamentales: el Modo Neural y el Modo Caos 3D.

1.2.1. Modo Neural: Redes Neuronales Productoras de Patrones Composicionales (CPPN)

El Modo Neural se aleja de la simulación iterativa en el tiempo para adentrarse en la generación paramétrica del espacio. En lugar de utilizar redes neuronales convencionales para clasificación o predicción, empleamos **Compositional Pattern Producing Networks (CPPNs)**.

Matemáticamente, una CPPN es una función continua universal que mapea coordenadas espaciales (y temporales) a valores de intensidad o de campo vectorial:

$$f : \mathbb{R}^n \rightarrow \mathbb{R}^m \quad (1.1)$$

donde el vector de entrada típico es $\mathbf{x} = (x, y, z, t)$ y la salida representa atributos físicos o visuales (densidad, velocidad, color).

La arquitectura interna de la red consiste en una composición de múltiples funciones de activación no lineales. Si definimos la salida de la capa i como h_i , el paso hacia adelante se formula como:

$$h_i = \phi_i(\mathbf{W}_i \cdot h_{i-1} + \mathbf{b}_i) \quad (1.2)$$

A diferencia de las redes tradicionales (que usan predominantemente ReLU o Sigmoide), las CPPN integran una amalgama de funciones de activación heterogéneas, incluyendo periódicas y radiales (seno, coseno, gaussianas, funciones de Bessel). Esta diversidad composicional permite que el mapeo resultante herede características estructurales específicas:

- La función seno ($\sin(x)$) introduce patrones repetitivos y frecuencias espaciales.
- La función gaussiana (e^{-x^2}) promueve simetría y contención espacial.
- La función identidad preserva gradientes lineales continuos.

La composición profunda de estas funciones permite que la red codifique imágenes de resolución teóricamente infinita y geometrías topológicamente complejas.

1.2.2. Modo Caos 3D: Dinámica No Lineal y el Atractor de Lorenz

Mientras que el Modo Neural mapea el espacio de forma analítica y continua, el Modo Caos 3D integra ecuaciones diferenciales ordinarias (EDOs) en el tiempo para estudiar la topología de los atractores extraños en el espacio de fase.

El pilar fundamental de este modo es el **Atractor de Lorenz**, un sistema tridimensional originalmente derivado de una simplificación de las ecuaciones de convección térmica de Navier-Stokes. Las ecuaciones que gobiernan el flujo están dadas por:

$$\frac{dx}{dt} = \sigma(y - x) \quad (1.3)$$

$$\frac{dy}{dt} = x(\rho - z) - y \quad (1.4)$$

$$\frac{dz}{dt} = xy - \beta z \quad (1.5)$$

Donde los parámetros físicos tienen un significado profundo:

- σ (Número de Prandtl): Relación entre la viscosidad cinemática y la difusividad térmica.
- ρ (Número de Rayleigh): Cuantifica la transferencia de calor por convección sobre la conducción. Superar el número crítico de Rayleigh desata la inestabilidad convectiva.
- β : Parámetro geométrico relacionado con el tamaño de las celdas convectivas.

Cuando $\rho > 24,74$ (con $\sigma = 10, \beta = 8/3$), el sistema converge a un atractor extraño fractal de dimensión Hausdorff aproximada a 2.06. La trayectoria orbita caóticamente alrededor de dos puntos de equilibrio inestables, oscilando impredeciblemente entre ellos. Este atractor subyace en la generación del caos visual en la simulación 3D.

1.3. Motores de Ecuaciones Diferenciales Parciales (EDPs)

Para la simulación rigurosa de fluidos y campos, *Cosmic-Generator* implementa un núcleo de ecuaciones diferenciales parciales que se resuelven utilizando métodos numéricos explícitos y análisis espectral.

1.3.1. Ecuación de Reacción-Difusión (Modelo de Gray-Scott)

El modelo de Gray-Scott simula reacciones químicas acopladas con la difusión de componentes en un sustrato. Constituye la base matemática para la generación de morfogénesis biológica y patrones de Turing.

Definimos dos especies químicas espaciotemporales, $U(\mathbf{r}, t)$ y $V(\mathbf{r}, t)$. Las dinámicas de concentración están dadas por:

$$\frac{\partial U}{\partial t} = D_u \nabla^2 U - UV^2 + F(1 - U) \quad (1.6)$$

$$\frac{\partial V}{\partial t} = D_v \nabla^2 V + UV^2 - (F + k)V \quad (1.7)$$

- D_u, D_v son los coeficientes de difusión. Es imperativo que $D_u > D_v$ para que surja la inestabilidad espacial de Turing.
- El término no lineal $\pm UV^2$ representa una reacción autocatalítica.
- F es la tasa de alimentación que repone la especie U , y k la tasa de decaimiento de V .

1.3.2. Ecuación de Kuramoto-Sivashinsky (KS)

La ecuación de KS es un modelo prototípico para las inestabilidades espaciotemporales en fluidos y frentes de llama, exhibiendo “caos espaciotemporal”.

$$\frac{\partial u}{\partial t} + \frac{1}{2} |\nabla u|^2 + \alpha \nabla^2 u + \gamma \nabla^4 u = 0 \quad (1.8)$$

Para su versión de perturbación en 1D:

$$\frac{\partial v}{\partial t} + v \frac{\partial v}{\partial x} + \alpha \frac{\partial^2 v}{\partial x^2} + \gamma \frac{\partial^4 v}{\partial x^4} = 0 \quad (1.9)$$

- $v \frac{\partial v}{\partial x}$: Transferencia de energía entre escalas (cascada no lineal).
- $\alpha \nabla^2 u$: Inyección de energía en grandes escalas, provocando inestabilidad local.

- $\gamma \nabla^4 u$: Hiper-amortiguación a altas frecuencias que estabiliza matemáticamente el sistema disipando la energía de las longitudes de onda muy cortas.

1.3.3. Ecuación de Gross-Pitaevskii (GPE)

Para el régimen cuántico macroscópico, empleamos la GPE. Modela la función de onda de un fluido cuántico a temperatura cercana al cero absoluto (Condensado de Bose-Einstein).

$$i\hbar \frac{\partial \psi(\mathbf{r}, t)}{\partial t} = \left(-\frac{\hbar^2}{2m} \nabla^2 + V_{\text{ext}}(\mathbf{r}, t) + g|\psi(\mathbf{r}, t)|^2 \right) \psi(\mathbf{r}, t) \quad (1.10)$$

- $\psi(\mathbf{r}, t)$ es la función de onda de orden de campo complejo.
- $-\frac{\hbar^2}{2m} \nabla^2$ es la dispersión de energía cinética de las partículas.
- $g|\psi|^2$ es la interacción de contacto no lineal. Si $g > 0$, es repulsiva (fluidos desenfocantes); si $g < 0$, atractiva (formación de solitones y colapso).

1.3.4. Ecuación de Onda

Finalmente, la ecuación clásica rige la propagación de campos sin dispersión ni no linealidades fuertes:

$$\frac{\partial^2 u}{\partial t^2} = c^2 \nabla^2 u \quad (1.11)$$

Donde c es la velocidad de propagación. Esta ecuación de naturaleza hiperbólica sirve como base para todo análisis acústico y electromagnético fundamental en nuestras simulaciones espaciales.

Parte II

Arquitectura Técnica y Audio

Capítulo 2

Arquitectura del Motor Reactivo: Mapeo de Audio y Dinámica Física

En este capítulo se desglosa el núcleo del sistema generativo, detallando el proceso de extracción de características musicales y su acoplamiento con sistemas de ecuaciones diferenciales parciales (PDEs). La correspondencia entre el espacio acústico y las variables de los modelos físicos permite que parámetros visuales emergentes —como la viscosidad, difusión o densidad de energía— reaccionen en tiempo real a estímulos sonoros.

2.1. Extracción de Descriptores de Audio

El módulo `audio_analyzer.py` utiliza la biblioteca `librosa` para deconstruir la señal acústica en componentes espectrales y rítmicos fundamentales, normalizados en el dominio $[0, 1]$ para alimentar el motor físico de renderizado:

- **Energía Percusiva y Armónica (RMS):** Se utiliza la separación armónico-percusiva (HPSS) y/o separación de tallos (Stems) por IA para derivar la energía cuadrática media (RMS). El golpe percusivo (**kick**) y la energía armónica (**harm**) son los principales conductores dinámicos de las deformaciones geométricas.
- **Centroide Espectral (*Spectral Centroid*):** Determina el "brillo" del sonido. Se mapea comúnmente a osciladores de alta frecuencia, transformaciones de matrices en fractales iterativos y rotaciones.

- **Chroma y Tonalidad Reina:** La detección de notas predominantes (C, C#, D...) regula las paletas de color mediante la cromestesia, tiñendo el multiverso físico según la firma tonal ("Global Note") o la nota dominante de cada instante temporal.

```

1 rms_perc = librosa.feature.rms(y=y_percussive, hop_length=
    hop_length)[0]
2 rms_perc = rms_perc / (np.max(rms_perc) + 1e-6)
3
4 cent = librosa.feature.spectral_centroid(y=y, sr=sr, hop_length=
    hop_length)[0]
5 cent = np.log1p(cent)
6 cent = (cent - np.min(cent)) / (np.max(cent) - np.min(cent) + 1e
    -6)

```

Listing 2.1: Extracción y normalización de descriptores sonoros en el módulo de audio.

2.2. Mapeo Sincro-Físico: De la Música a la Ecuación

Dentro del orquestador `render_standard.py`, el motor traduce el vector de características de cada *frame* a fuerzas físicas directas que modulan los algoritmos iterativos.

2.2.1. Paso de Tiempo Dinámico (CFL Control)

Para mantener simulaciones altamente reactivas pero robustas, el incremento de tiempo dt es ajustado dinámicamente. Al estar inversamente escalado por la intensidad percusiva, actúa como un limitador temporal adaptativo (inspirado en el criterio de Courant-Friedrichs-Lewy) para prevenir la inestabilidad numérica que podría causar la inyección abrupta de energía acústica al motor físico.

```

1 # Si hay mucho caos (kick alto), reducimos el paso de tiempo
    para estabilidad
2 dt_dynamic = 1.0 / (1.0 + kick * 2.0)

```

Listing 2.2: Paso de tiempo dinámico modulado por la percusión (`kick`).

2.2.2. Modulación del Modelo Gray-Scott (Turing)

El patrón de reacción-difusión de Gray-Scott se rige por la tasa de alimentación (*feed rate*, f) y la tasa de muerte (*kill rate*, k). Se establece un mapeo biológico: la energía armónica (**harm**) favorece la proliferación de la especie V .^aumentando la tasa de alimentación, mientras que la percusiva (**kick**) reduce su muerte prematura, resultando en estallidos y expansiones de manchas orgánicas con los *beats*.

```

1 # harm aumenta la tasa de alimentacion (feed f)
2 f = escena['p1'] + (harm * 0.01)
3 # kick reduce la tasa de muerte celular (kill k)
4 k = escena['p2'] - (kick * 0.005)

```

Listing 2.3: Mapeo acústico a parámetros del sistema de reacción-difusión.

2.2.3. Modulación de Ondas y Otras PDEs

El bajo profundo (*Bass RMS*), mapeado a la variable de textura, altera la dinámica topológica (por ejemplo, el coeficiente de movilidad de Cahn-Hilliard). En el caso de la ecuación de ondas (WAVE), además de modular el factor de propagación c^2 , los eventos percusivos severos funcionan como perturbaciones físicas inyectando "gotas." impulsos iniciales en el campo:

```

1 c2 = escena['p2'] * dt_dynamic
2 # Inyectar gotas con el audio
3 if kick > 0.4 or harm > 0.6:
4     ry, rx = np.random.randint(1, gs_h-1), np.random.randint(1,
5         gs_w-1)
6     wave_u[ry, rx] += np.random.uniform(-1, 1) * kick

```

Listing 2.4: Inyección de eventos puntuales al campo escalar de ondas.

2.3. Estabilidad Numérica y Limitadores Anti-Clipping

Los sistemas no lineales alimentados asíncronamente con datos de la vida real (ruido o música) tienden frecuentemente a divergir y generar desbordamientos (*overflows*). Para garantizar un *render* estable sin sacrificar reactividad, se emplean métodos de limitación denominados "Homeostasis".

Estas rutinas de contención detectan umbrales críticos o valores NaN en matrices (como el espacio u de Korteweg-de Vries o Kuramoto-Sivashinsky) y responden mitigando, atenuando o reiniciando la energía inyectada:

```

1 # HOMEOSTASIS WAVE: Limitador anti-clipping del campo tensorial
2 if np.max(np.abs(wave_u)) > 50.0:
3     wave_u *= 0.5
4     wave_u_prev *= 0.5 # Correccion fisica para no perder la
      inercia
5
6 # HOMEOSTASIS KS (Kuramoto-Sivashinsky)
7 if np.max(np.abs(ks_u)) > 1e3 or np.any(np.isnan(ks_u)):
8     ks_u = np.zeros_like(ks_u) + np.random.normal(0, 0.1, ks_u.
      shape)
9
10 # Sanitizacion estricta contra propagacion de NaNs
11 ks_u = np.nan_to_num(ks_u, nan=0.0, posinf=10.0, neginf=-10.0)

```

Listing 2.5: Mecanismos de Homeostasis para evitar rupturas del solucionador PDE.

2.4. Generación Estocástica y Uso de Semillas Aleatorias

En sistemas caóticos donde participan atractores matemáticos (como Clifford o Lorenz) y autómatas celulares, una divergencia minúscula en el estado inicial produce ramificaciones topológicas drásticamente asimétricas.

La inserción parametrizada de semillas pseudoaleatorias (*Random Seeds*) obedece a dos necesidades principales en el proyecto:

1. **Reproducibilidad Artística:** Un músico o ingeniero que compila el mismo archivo de audio introduciendo la misma `seed` requiere la garantía de generar el exacto mismo flujo visual iterativo. Esto es fundamental para los ciclos iterativos de testeo y control de calidad VJ.
2. **Diferenciación de Entidades Autónomas:** Al instanciar múltiples entidades matemáticas (`EspirituProcedural`), se suministran semillas algorítmicamente desfasadas (`seed_val=i*100`). Esto fuerza la independencia de las ecuaciones paramétricas, obligándolas a recorrer órbitas ortogonales para no solaparse en el render final y maximizando de esta forma la entropía y el volumen visual de la pantalla.

```

1 if seed is not None:
2     print(f"Usando Seed: {seed}")

```

```

3     np.random.seed(seed) # Fuerza una cascada determinista
4
5 # Espíritus inicializados con trayectorias caóticas divergentes
6 espíritus = [EspirituProcedural(WIDTH, HEIGHT, seed_val=i*100)
7               for i in range(num_espíritus)]

```

Listing 2.6: Uso de semillas en el núcleo de aleatoriedad general y particular.

2.5. Conclusión del Mapeo

El puente arquitectónico desarrollado entre las rutinas analíticas de **librosa** y los solucionadores *hyper-threaded* genera una estricta "sinestesia matemática". El estímulo acústico no se utiliza arbitrariamente para animar elementos genéricos, sino que se inyecta de forma basal dentro del subsuelo numérico (tal como la modulación del potencial cuántico en Gross-Pitaevskii o el umbral de atenuación en las ondas). Esto permite al espectador discernir el carácter, densidad temporal, grosor frecuencial y texturizado físico de cada pista de audio en un medio fluido enteramente sintético.

Parte III

El Nexo Central

Sinergia: Física + Programación

La Filosofía de la Rebeldía Cósmica

En el corazón de la Cooperativa *Rebeldía Cósmica* yace una convicción fundamental: el arte y la ciencia no son dominios aislados, sino lenguajes complementarios que, al entrelazarse, revelan la belleza subyacente del universo. El **Cosmic-Generator** es la materialización de esta filosofía, una herramienta diseñada para democratizar la creación artística, permitiendo que cualquier individuo, sin importar su origen, pueda tejer tapices audiovisuales a partir del tejido mismo de la realidad.

El Alma: Ecuaciones Puras (Física)

La física es el estudio de las reglas que rigen el cosmos. En el contexto de nuestro generador, las ecuaciones puras actúan como el pincel y los pigmentos.

- **Ondas y Frecuencias:** Cada sonido que procesamos es, en su esencia, una perturbación mecánica, una suma de sinusoides descritas por la Transformada de Fourier. Las matemáticas de las ondas sonoras ($y(t) = A \sin(\omega t + \phi)$) se convierten en las semillas de la geometría visual.
- **Dinámica y Fuerzas:** Inspirándonos en la mecánica celeste y la dinámica de fluidos, las partículas y los colores en pantalla no se mueven de forma aleatoria, sino que obedecen a leyes análogas a la gravedad y el electromagnetismo, creando un caos estructurado y orgánico.

El Músculo: Arquitectura y Extracción (Programación)

Una filosofía brillante requiere un motor robusto. Aquí es donde la ingeniería de software interviene para transformar la teoría en una experiencia viva y en tiempo real.

- **Arquitectura de Hilos (Multithreading):** Para que el arte fluya sin interrupciones, el sistema divide su mente. Un hilo captura vorazmente las ondas del micrófono o archivo de audio, mientras otro procesa matemáticamente la información y un tercero dibuja los fractales o formas reactivas en la pantalla. Esta asincronía garantiza que el cálculo pesado no paralice el flujo cósmico de los visuales.
- **Extracción de Características:** Mediante algoritmos avanzados (como el cálculo del Centroide Espectral, MFCCs y RMS), extraemos la *emoción* oculta en los datos en crudo. La programación traduce la intensidad acústica en brillo, y el tono en paletas de colores, actuando como un puente empático entre el fenómeno físico y la percepción humana.

La Fusión: El Arte del Cosmic-Generator

La verdadera sinergia ocurre cuando el hilo de ejecución lee la variable de amplitud sonora (física) y reajusta la matriz de transformación de la GPU (programación). Es en este preciso instante donde el código y las matemáticas pierden su frialdad y se convierten en **Arte**.

El *Cosmic-Generator* no es solo un programa; es un manifiesto. Democratizamos el arte al empaquetar la inmensidad de las matemáticas y la complejidad de la ingeniería de software en una interfaz donde tú eres el director de orquesta. Al liberar la creación audiovisual de las élites tecnológicas, empoderamos a cada usuario para explorar su propia *Rebeldía Cósmica*.

Parte IV

Legado: Manual Volumen 1

Parte V

Arquitectura del Sistema

2.6. Arquitectura del Código y Flujo de Trabajo

La arquitectura del simulador físico Cosmic-Generator está estructurada para maximizar la modularidad, el rendimiento y la facilidad de uso. El flujo de trabajo o pipeline sigue un modelo secuencial desde la captura de parámetros del usuario hasta la visualización en tiempo real y la síntesis de audio.

2.6.1. Punto de Entrada: UI y Configuración (`app.py`)

El ciclo de ejecución de la plataforma comienza en el archivo `app.py`. Este script sirve como punto de entrada de la aplicación y utiliza el framework Streamlit para proveer una Interfaz de Usuario (UI) web interactiva. En esta etapa, `app.py` permite al usuario definir las variables globales y parámetros específicos del sistema (tales como constantes físicas, niveles de fricción, propiedades visuales, entre otros). Al iniciar la simulación, los parámetros se empaquetan y se transmiten al siguiente nivel de la arquitectura.

2.6.2. Orquestación del Sistema (`lanzador.py`)

El módulo `lanzador.py` cumple el rol de orquestador o controlador principal. Recibe los parámetros configurados en la UI e inicializa todas las piezas fundamentales del simulador. Sus responsabilidades incluyen:

- Instanciar el entorno físico.
- Configurar los subsistemas de generación de audio y renderizado visual.
- Disparar el bucle principal de simulación, delegando la carga computacional a los módulos especializados.

De esta forma, se mantiene completamente desacoplada la interfaz web de la lógica pesada del simulador.

2.6.3. Patrón Productor-Consumidor en Renderizado (`src/render/stable`)

El rendimiento es crítico en simulaciones físicas de tiempo real. Para asegurar una alta tasa de fotogramas por segundo (FPS), el archivo `src/render/stable/render_standard.py` implementa un modelo de concurrencia basado en el patrón **Productor-Consumidor**, empleando múltiples hilos (threads):

- **Productor (Hilo Físico):** Un hilo en segundo plano se dedica exclusivamente a resolver las ecuaciones matemáticas del simulador. Calcula la cinemática, actualiza posiciones, velocidades y detecta colisiones, produciendo *estados del sistema* que se almacenan de forma segura en una estructura de datos concurrente (cola o buffer).
- **Consumidor (Hilo Gráfico):** El hilo principal de renderizado lee o consume los estados físicos ya calculados que están disponibles en el buffer, para dibujar cada fotograma. Al separar estos dos procesos, el cálculo matemático no bloquea el repintado de la pantalla y la visualización gráfica no ralentiza la física, logrando una aceleración notable de los FPS.

2.6.4. Organización Modular del Código

El código fuente se organiza de forma rigurosa y cohesiva en distintos directorios bajo `src/`, respetando el principio de responsabilidad única:

- **src/core/:** Contiene el motor numérico y físico del proyecto. Aquí residen los cálculos de dinámica gravitacional, cinemática y detección de colisiones. Este módulo opera con matemática pura y no tiene conocimiento del entorno gráfico ni sonoro.
- **src/audio/:** Módulo encargado de la síntesis y gestión de sonido. Actúa escuchando los eventos disparados por el Core (por ejemplo, el rebote o impacto de una entidad) y los transforma dinámicamente en efectos de audio espaciales o arreglos musicales.
- **src/render/:** Dedicado de forma exclusiva a la representación visual. Recopila la información geométrica del Core y realiza las llamadas a la biblioteca gráfica pertinente para renderizar el entorno y los elementos simulados en la ventana de la aplicación.

2.7. Módulo: src/core (Motor Numérico y Físico)

El directorio `src/core` constituye el cerebro matemático de la simulación. Su responsabilidad es pura y exclusivamente el cálculo numérico, la dinámica geométrica y la actualización de los estados físicos del universo, permaneciendo completamente ciego respecto a cómo estos datos se dibujan en pantalla o cómo suenan.

2.7.1. Archivos Principales y Funcionalidad

- **nucleo_espectral.py** y **nucleo_neural.py**: Estos archivos contienen las clases matemáticas que resuelven las ecuaciones diferenciales (como Gray-Scott o Navier-Stokes) y las redes neuronales (CPPN). Manejan tensores de NumPy/PyTorch para calcular gradientes espaciales y operadores laplacianos en cada iteración temporal.
- **visual_entities.py**: Define la orientación espacial, vectores de posición, velocidad y aceleración de las partículas discretas en simulaciones de N-cuerpos.
- **efectos_visuales.py**: Gestiona las transformaciones proyectivas de la matriz espacial, tales como las rotaciones en ejes 3D proyectadas a 2D y el factor de escala (zoom) dictaminado por la intensidad física calculada.

2.7.2. Cálculos de Dinámica y Cinemática

En este módulo, el tiempo se trata como una variable discreta. En cada actualización (*update*), las coordenadas y matrices de estado evolucionan. Se resuelven integraciones explícitas de Euler o Runge-Kutta para asegurar que la masa, el momento y la energía matemática se comporten bajo leyes deterministas. No existen colores ni píxeles aquí, sólo arreglos de números flotantes de alta precisión.

2.8. Módulo: `src/audio` (Síntesis y Gestión de Sonido)

El directorio `src/audio` es el responsable de actuar como el puente sensorial de la simulación. En Cosmic-Generator, el flujo de información es bidireccional: la música altera la física, pero la física (eventos) también debe ser interpretada.

2.8.1. Extracción Acústica (`audio_analyzer.py`)

Este archivo emplea la librería `librosa` para diseccionar un archivo de audio de entrada en variables numéricas utilizables por el motor físico. Extrae:

- **Energía (RMS)**: La amplitud o volumen del fotograma actual.
- **Spectral Flux (Flujo Espectral)**: Los ataques transitorios (ej: golpes de batería o platillos).

2.9. MÓDULO: SRC/RENDER (REPRESENTACIÓN VISUAL EXCLUSIVA)²⁵

- **Tonalidad y Pitch Centroide:** Las frecuencias fundamentales que definirán las paletas cromáticas (*Cromesthesia*).

2.8.2. Alineación de Letras (`motor_lyrics.py`)

Contiene el motor de Inteligencia Artificial (`stable-ts` / Whisper). Escucha el archivo de audio, aísla las frecuencias vocales y genera un diccionario JSON con las marcas de tiempo (timestamps) exactas de cada palabra. Además, implementa la tecnología de **Alineación Forzada**, permitiendo al usuario inyectar la letra original y obligando a la IA a sincronizar matemáticamente las sílabas, eliminando cualquier tipo de alucinación del modelo.

2.8.3. Eventos Disparados por el Core

Cuando el módulo físico (`src/core`) detecta una colisión de alta energía, envía una señal asíncrona a este módulo. Dependiendo de la magnitud del impacto, el sistema de audio puede inyectar un efecto sonoro o alterar la retroalimentación visual basada en frecuencias sonoras inyectadas de vuelta al pipeline.

2.9. Módulo: `src/render` (Representación Visual Exclusiva)

El directorio `src/render` es el final de la cadena de montaje computacional (Pipeline). Es el módulo consumidor puro: no realiza cálculos físicos ni análisis de audio; su única misión es recopilar los tensores numéricos del `core`, asignarles texturas y enviarlos a la pantalla o a un archivo de video final de manera ultrarrápida.

2.9.1. Estructura del Motor de Renderizado

El directorio se divide en ramas de estabilidad:

- **`stable/render_standard.py`:** El bucle principal de renderizado usando OpenCV (`cv2`). Se encarga de transformar las matrices abstractas del `core` en fotografías RGB (*frames*).
- **`experimental/`:** Contiene motores de renderizado más avanzados o no lineales (como trazado de rayos básico o Shaders nativos) que están en fase beta.

2.9.2. Proceso de Dibujado (Drawing Pipeline)

El motor `render_standard.py` orquesta la función `generar_animacion_god_mode()`. Su ciclo de vida es el siguiente:

1. Lee la matriz escalar generada por `src/core` (por ejemplo, el mapa de densidad de un fluido).
2. Aplica un mapeo de color (`Colormap`) dinámico basado en las variables dictadas por `src/audio` (Tonalidad).
3. Si corresponde, dibuja las partículas usando operaciones de trazado matricial o `cv2.circle`.
4. Si el `motor_lyrics.py` envía texto sincronizado, invoca rutinas tipográficas (`Pillow` o `cv2.putText`) para superimprimir la lírica en pantalla con colores vibrantes y desenfoques (`Bloom`).
5. Escribe el fotograma directamente en el *Video Writer* (`FFmpeg`) para exportar el ‘mp4’ final, reportando el progreso a la interfaz de usuario.

Parte VI

El Puente Sensorial (Audio a Física)

2.10. Extracción de Parámetros de Audio

El corazón rítmico de Cosmic-Generator reside en su módulo de análisis avanzado, ubicado en `src/audio/motor_audio.py`. Para lograr una reactividad visual ultra optimizada y matemáticamente precisa, empleamos la librería `librosa` para diseccionar el espectro acústico en el dominio del tiempo y la frecuencia, transformándolo en tensores de control que alimentan directamente nuestra tubería gráfica.

2.10.1. Energía y Zoom (RMS)

El cálculo del *Root Mean Square* (RMS) nos proporciona una métrica directa de la energía térmica y amplitud de la señal. Extraemos la energía promedio por cada *frame* de audio y la aplicamos como un escalar no lineal en la matriz de transformación proyectiva (Zoom). De esta manera, a mayor intensidad percusiva o volumen acústico, generamos una compresión espacial que empuja la cámara hacia la acción.

2.10.2. Turbulencia y Flujo Espectral (Spectral Flux)

Para inducir caos controlado e inestabilidades realistas en la simulación de partículas y fluidos, calculamos el *Spectral Flux*. Esta métrica evalúa la tasa de cambio (la primera derivada discreta) de la magnitud del espectro de potencias entre *frames* consecutivos. Este gradiente espectral alimenta de inmediato nuestros campos vectoriales y los coeficientes de advección, inyectando vórtices y ráfagas de turbulencia direccional proporcionales a los ataques e impulsos de alta frecuencia de la pista.

2.10.3. Cromestesia y Color (Pitch/Tonalidad)

Nuestra arquitectura de sombreadores de fragmentos (*Fragment Shaders*) no emplea paletas de colores estáticas. El color de la simulación obedece a un mapeo dinámico entre el *pitch* dominante y el espacio cromático. Mediante `librosa`, extraemos la frecuencia fundamental (f_0) y el cromagrama del audio para clasificar la tonalidad en tiempo real. Esta tonalidad modula los canales RGB de forma algorítmica, induciendo una cromestesia donde las frecuencias graves tiñen el entorno de fluidos densos y rojizos, mientras que los armónicos agudos cristalizan la escena en destellos cianes y blancos de alta luminancia.

2.11. Frecuencia y Sincronización de Cambio de Parámetros

La sincronización milimétrica entre la integración del motor físico en la GPU y la línea de tiempo acústica es nuestro mayor desafío resuelto. Hemos estructurado una arquitectura asíncrona altamente paralela para evitar bloqueos y asegurar que el renderizado se mantenga fluido sin sacrificar la rigurosidad de las colisiones y campos de fuerza.

2.11.1. Muestreo Discreto del Audio

En la fase de pre-cómputo, el módulo de audio procesa la pista completa extrayendo características *frame a frame*. Estableciendo una frecuencia de muestreo de fotogramas (por ejemplo, a 30 FPS constantes), dividimos el audio obteniendo un arreglo vectorial de tamaño $\text{duración total} \times \text{FPS}$. Este arreglo indexable nos permite un acceso a memoria en $\mathcal{O}(1)$ para extraer las variables ambientales necesarias de acuerdo al tiempo de la simulación.

2.11.2. Time-stepping Loop vs. Render Loop

Para maximizar el rendimiento gráfico, implementamos un desacoplamiento de bucles. El bucle de dibujado (Render Loop) está dedicado exclusivamente a interpolar y enviar *buffers* a la pantalla con la menor latencia posible. Simultáneamente, en un hilo de ejecución paralelo, transcurre el *Time-stepping Loop* de la simulación física. En CADA iteración de este bucle físico, el motor extrae el *timestamp* actual e indexa el *array* de audio, inyectando los parámetros del *frame* acústico exactamente en el momento que la física requiere su actualización.

2.11.3. Mutación Dinámica de la Difusión

El verdadero poder del Cosmic-Generator se despliega en cómo la acústica altera las leyes físicas. En cada actualización de los *solvers* de la simulación de fluidos, los coeficientes de las ecuaciones de difusión mutan drásticamente: la intensidad acústica (derivada del RMS) modula escalarmente la viscosidad y difusividad térmica, forzando a los fluidos a volverse más reactivos o densos. Simultáneamente, las frecuencias dominantes inyectan anisotropía direccional al tensor de difusión. El resultado es un estado de materia puramente reactivo, donde el sonido altera topológicamente los coeficientes matemáticos a razón de 30 veces por segundo,

esculpiendo gráficamente la canción.

2.12. El Puente Físico-Sonoro: Destruyendo la Caja Negra

Este documento revela el mecanismo exacto, a nivel matemático y de código, mediante el cual las señales de audio se inyectan en los cinco motores físicos del Cosmic-Generator. No hay magia ni cajas negras; hay un mapeo directo y determinista entre el análisis de la señal y las ecuaciones diferenciales parciales.

2.12.1. Extracción de Parámetros de Audio

Para cada fotograma (típicamente a 30 o 60 FPS), el sistema procesa una ventana de la señal de audio de entrada y extrae un vector de características espectrales y temporales. Estas variables de control se normalizan y se aplican filtros de suavizado antes de integrarse en los métodos numéricos.

- **Amplitud (RMS) – $A(t)$:** Representa la energía global instantánea. Controla variables macroscópicas, inyecciones de energía de fondo y tasas de alimentación.
- **Flujo Espectral (Spectral Flux) – $S(t)$:** Mide los cambios abruptos de energía entre frames (transitorios, golpes de percusión). Se usa para inyectar perturbaciones impulsivas y forzamientos abruptos.
- **Tonalidad / Centroide Espectral – $T(t)$:** Representa el ‘color’ o brillo del sonido. Modula propiedades intrínsecas del medio, como coeficientes de difusión, repulsión o velocidad de onda.

2.12.2. Motor 1: Reacción-Difusión (Gray-Scott / Corrosión)

El motor Gray-Scott simula la interacción química de dos sustancias, u y v , generando laberintos, células y patrones de Turing. Las ecuaciones que evolucionamos son:

$$\begin{aligned}\frac{\partial u}{\partial t} &= D_u \nabla^2 u - uv^2 + f(t)(1 - u) \\ \frac{\partial v}{\partial t} &= D_v \nabla^2 v + uv^2 - (f(t) + k(t))v\end{aligned}$$

Inyección de Audio: Las constantes f y k , que en la formulación original dictan la ‘especie’ del patrón, mutan dinámicamente con el audio:

- **Tasa de Alimentación (Feed Rate, f):** La amplitud $A(t)$ incrementa la tasa f . Un sonido denso introduce más químico u , hinchando los patrones hasta volverlos expansivos e interconectados.

$$f(t) = f_{base} + \alpha_f A(t)$$

- **Tasa de Mortalidad (Kill Rate, k):** Se perturba negativamente con los golpes rítmicos ($S(t)$). Un ‘kick’ disminuye momentáneamente k , provocando explosiones celulares rápidas.

$$k(t) = k_{base} - \alpha_k S(t)$$

2.12.3. Motor 2: Kuramoto-Sivashinsky (Fuego / Caos Espacial)

Este motor modela inestabilidades de propagación de frentes (como llamas) exhibiendo turbulencia espacio-temporal. La ecuación escalar modificada es:

$$\frac{\partial u}{\partial t} + \nu \nabla^4 u + \mu(t) \nabla^2 u + \frac{1}{2} |\nabla u|^2 = F(x, y, t) \quad (2.1)$$

Inyección de Audio:

- **Coefficiente de Inestabilidad (μ):** La amplitud o intensidad de los graves aumenta el coeficiente μ , inyectando energía en longitudes de onda largas y provocando que el ‘fuego’ se vuelva más turbulento y desordenado.

$$\mu(t) = \mu_0 + \alpha_\mu A(t)$$

- **Forzamiento Espacial (F):** Los transitorios $S(t)$ activan mapas de ruido espacial en instantes puntuales, actuando como inyecciones o ‘chispas’ que alimentan el campo escalar localizado, perturbando la ecuación bruscamente.

2.12.4. Motor 3: Gross-Pitaevskii (Cuántico / Superfluido)

Describe el comportamiento microscópico de un condensado de Bose-Einstein o de un superfluido, modelado sobre una función de onda compleja ψ .

$$i\hbar \frac{\partial \psi}{\partial t} = \left(-\frac{\hbar^2}{2m} \nabla^2 + V(\mathbf{x}, t) + g(t)|\psi|^2 \right) \psi \quad (2.2)$$

Inyección de Audio:

- **Potencial Externo (V):** El audio genera un campo topológico sobre el lienzo. La intensidad global $A(t)$ modula la profundidad o barreras del potencial atrapando o liberando el fluido cuántico en zonas de alta densidad energética visual.
- **Constante de Auto-Interacción (g):** Se asocia a la Tonalidad o Centroide $T(t)$. Si el sonido tiene frecuencias predominantemente agudas, el coeficiente g se vuelve más repulsivo (o menos atractivo), alterando el tamaño y la velocidad de nucleación de los vórtices cuánticos.

2.12.5. Motor 4: Onda Clásica

Simula la propagación de deformaciones transversales en una malla 2D ideal (análogo al parche de un tambor).

$$\frac{\partial^2 u}{\partial t^2} = c(t)^2 \nabla^2 u - \gamma(t) \frac{\partial u}{\partial t} + P(\mathbf{x}, t) \quad (2.3)$$

Inyección de Audio:

- **Velocidad de Propagación (c):** Se vuelve una función dinámica ligada a la frecuencia fundamental del audio. Sonidos más agudos ‘tensan’ el medio virtual, acelerando la velocidad de fase c .
- **Amortiguamiento (γ):** La amplitud sostenida reduce el rozamiento interno. Un drone musical provoca que las ondas perduren en el espacio; el silencio incrementa el amortiguamiento limpiando el lienzo.
- **Excitación (P):** Golpes rítmicos mapean fuerza pura en coordenadas específicas, emitiendo verdaderas frentes de onda en sincronía.

2.12.6. Motor 5: Korteweg-de Vries (KdV / Tsunamis, Solitones)

La ecuación KdV (o su extensión 2D Kadomtsev-Petviashvili) permite la emergencia de solitones, donde el empujamiento no lineal balancea exactamente la dispersión de las ondas.

$$\frac{\partial u}{\partial t} + \alpha(t)u \frac{\partial u}{\partial x} + \beta(t) \frac{\partial^3 u}{\partial x^3} = 0 \quad (2.4)$$

Inyección de Audio:

- **Efecto No Lineal (α):** Está directamente controlado por la energía RMS de la señal de audio. Al incrementarse el volumen general, α crece desproporcionadamente, provocando que los frentes de onda se empujen formando picos afilados o estructuras tipo ‘tsunami’.

$$\alpha(t) = \alpha_{base} \cdot (1 + \lambda_A A(t))$$

- **Efecto Dispersivo (β):** Responde al perfil espectral $T(t)$. Frecuencias muy agudas y brillantes aumentan el término dispersivo diferencial, dividiendo grandes frentes de onda en un tren múltiple de solitones de alta frecuencia.

Conclusión: En Cosmic-Generator, el audio no actúa como un mero filtro visual en etapa de post-procesamiento. Las propiedades rítmicas, dinámicas y armónicas de la música se convierten en perturbaciones, coeficientes no lineales y tasas de propagación insertadas directamente en los integradores Euler, Runge-Kutta de 4to orden y esquemas de Diferencias Finitas (FDTD). La música rediseña las leyes físicas del universo por cada Δt .

Parte VII

Motores Físicos y Ecuaciones Diferenciales

2.13. Fundamentos Físico-Matemáticos de los Motores Dinámicos

En esta sección, formalizamos los modelos fenomenológicos que subyacen a los diversos motores visuales del simulador *Cosmic-Generator*. La rigurosidad nos exige comprender que los patrones observados no son meros artefactos gráficos, sino soluciones emergentes de sistemas de Ecuaciones Diferenciales Parciales (EDP) altamente no lineales.

2.13.1. Motor de Corrosión: Modelo de Reacción-Difusión de Gray-Scott

El motor que produce los intrincados patrones biológicos y de corrosión”se basa en el célebre sistema de reacción-difusión introducido por P. Gray y S.K. Scott. Este modelo describe la cinética de dos especies químicas, U y V , que se difunden en el espacio y reaccionan según una dinámica de retroalimentación autocatalítica. El sistema acoplado se rige por:

$$\frac{\partial u}{\partial t} = D_u \nabla^2 u - uv^2 + f(1 - u) \quad (2.5)$$

$$\frac{\partial v}{\partial t} = D_v \nabla^2 v + uv^2 - (f + k)v \quad (2.6)$$

donde $u(\mathbf{r}, t)$ y $v(\mathbf{r}, t)$ representan las concentraciones espaciales. D_u y D_v son las constantes de difusión, f es la tasa de alimentación (feed) que repone U , y k la tasa de decaimiento o aniquilación (kill) de V . Los patrones visuales emergen de la inestabilidad de Turing inducida por la competencia entre una difusión rápida de la especie inhibidora y la reacción local de la especie activadora.

2.13.2. Motor de Fuego: Ecuación de Kuramoto-Sivashinsky

Para la simulación de frentes de llama y turbulencia espaciotemporal, empleamos la ecuación de Kuramoto-Sivashinsky. Originalmente derivada para modelar inestabilidades termo-difusivas en frentes de combustión, su forma fenomenológica en términos de fluctuaciones del frente $h(\mathbf{r}, t)$ es:

$$\frac{\partial h}{\partial t} = -\frac{1}{2}|\nabla h|^2 - \nabla^2 h - \nabla^4 h \quad (2.7)$$

Aquí observamos tres contribuciones fundamentales: el término no lineal $\frac{1}{2}|\nabla h|^2$

que estabiliza el crecimiento y transfiere energía entre escalas; el término difusivo inverso $-\nabla^2 h$ que bombea energía al sistema (inestabilidad en bajas frecuencias); y el término hiperdifusivo $-\nabla^4 h$ que disipa las perturbaciones de onda corta. Esto genera el comportamiento caótico y fractal propio del "fuego".

2.13.3. Motor Cuántico: Ecuación de Gross-Pitaevskii

Para representar campos de naturaleza cuántica, como los condensados de Bose-Einstein o fluidos de luz, recurrimos a una formulación de campo medio que obedece a la Ecuación de Schrödinger No Lineal (NLS), conocida en materia condensada como ecuación de Gross-Pitaevskii:

$$i\hbar \frac{\partial \psi}{\partial t} = \left(-\frac{\hbar^2}{2m} \nabla^2 + V(\mathbf{r}) + g|\psi|^2 \right) \psi \quad (2.8)$$

donde $\psi(\mathbf{r}, t)$ es la función de onda macroscópica compleja, m es la masa efectiva, $V(\mathbf{r})$ es un potencial externo de confinamiento, y el término $g|\psi|^2$ representa las interacciones de contacto entre partículas (repulsivas para $g > 0$). Las fluctuaciones en su amplitud y fase dan origen a vórtices cuánticos y a una interferencia dinámica soberbia.

2.13.4. Motor de Onda: Dinámica Hiperbólica Clásica

El motor de ondulaciones superficiales es, en el fondo, una integración de la ecuación de onda clásica, el arquetipo de las ecuaciones diferenciales hiperbólicas:

$$\frac{\partial^2 u}{\partial t^2} = c^2 \nabla^2 u - \gamma \frac{\partial u}{\partial t} \quad (2.9)$$

El campo escalar $u(\mathbf{r}, t)$ representa la elongación respecto al equilibrio. El parámetro c denota la velocidad de propagación de la perturbación, y hemos añadido un término fenomenológico $\gamma \partial_t u$ para modelar la disipación (amortiguamiento). La rigurosa conservación de la energía y el principio de Huygens se manifiestan visualmente a través de la reflexión y refracción.

2.13.5. Motor de Tsunamis: Ecuación de Korteweg-de Vries (KdV)

Finalmente, la asombrosa persistencia de ondas solitarias en el entorno visual está comandada por la ecuación de Korteweg-de Vries (KdV). Modelando aguas poco profundas, nos entrega la génesis de los *solitones*:

$$\frac{\partial u}{\partial t} + \alpha u \frac{\partial u}{\partial x} + \beta \frac{\partial^3 u}{\partial x^3} = 0 \quad (2.10)$$

Esta maravilla matemática describe cómo la dispersión lineal (el término con tercera derivada $\partial_x^3 u$) y el empujamiento no lineal (el término advectivo $u \partial_x u$) se balancean con exactitud. El resultado es un paquete de ondas que se propaga sin deformarse temporalmente, un fenómeno crítico para la descripción de los tsunamis y ciertas fibras ópticas.

2.14. Discretización y Simulación Numérica de Sistemas Dinámicos

Es un principio innegable en la física computacional moderna que una Ecuación Diferencial Parcial carece de utilidad empírica hasta que no es llevada a un entorno discreto estable. En *Cosmic-Generator*, materializamos las teorías del Capítulo 1 utilizando tensores en PyTorch o arreglos contiguos en NumPy, acelerados frecuentemente mediante el compilador JIT Numba. El éxito de estos métodos recae en la aplicación canónica de Diferencias Finitas y algoritmos de integración temporal sólidos.

2.14.1. Discretización Espacial y el Operador Laplaciano

El núcleo de la mayoría de nuestros motores (Gray-Scott, Onda, Kuramoto-Sivashinsky) depende de la evaluación precisa del Laplaciano espacial $\nabla^2 u$. En un retículo bidimensional de espaciado uniforme $\Delta x = \Delta y$, aplicamos un esquema de diferencias finitas de segundo orden (la plantilla de 5 puntos de Von Neumann):

$$\nabla^2 u_{i,j} \approx \frac{u_{i+1,j} + u_{i-1,j} + u_{i,j+1} + u_{i,j-1} - 4u_{i,j}}{(\Delta x)^2} \quad (2.11)$$

Para las evaluaciones dispersivas o de mayor orden, como el término bilaplaciano $\nabla^4 h$ en Kuramoto-Sivashinsky o la tercera derivada en KdV, se componen iterativamente estas aproximaciones o se aplican plantillas extendidas de 9 o más puntos para preservar la simetría rotacional en la grilla y evitar artefactos anisotrópicos puramente computacionales.

2.14.2. Condiciones de Frontera Periódicas y Optimizaciones Tensoriales

Un simulador continuo precisa una topología sin bordes para evitar reflexiones espurias, asumiendo una geometría toroidal. Esto lo logramos matemáticamente imponiendo condiciones de frontera periódicas: $u(0, y) = u(L_x, y)$ y $u(x, 0) = u(x, L_y)$.

A nivel de software (NumPy o PyTorch), esto se vectoriza magistralmente mediante la operación `roll`. En lugar de utilizar lentos bucles `for`, desplazamos el estado global del sistema de esta manera:

$$u_{i+1,j} \implies \text{np.roll}(u, \text{shift}=-1, \text{axis}=0) \quad (2.12)$$

Sumar matrices desplazadas permite computar las diferencias finitas sobre toda la malla simultáneamente, sacando provecho de las instrucciones SIMD del procesador o la paralelización extrema de las arquitecturas CUDA en la GPU.

2.14.3. Integración Temporal: Euler Explícito vs. Runge-Kutta

Toda vez calculado el funcional de derivadas espaciales $F(u)$, la evolución se transforma en un problema de valor inicial: $\partial_t u = F(u)$. Para los motores que demandan máximo rendimiento en tiempo real y exhiben alta disipación (como Gray-Scott o la Onda clásica), utilizamos el método de **Euler explícito**, el cual aproxima la serie de Taylor al primer orden:

$$u(t + \Delta t) = u(t) + \Delta t \cdot F(u(t)) \quad (2.13)$$

Aunque computacionalmente económico, el método de Euler exige un paso temporal Δt severamente restringido por el criterio de estabilidad de Courant-Friedrichs-Lewy (CFL), especialmente ante los términos difusivos ($\Delta t \propto (\Delta x)^2$).

Por el contrario, para ecuaciones rígidas ("stiff") y sin disipación natural como la Ecuación de Schrödinger (Gross-Pitaevskii) o la KdV, el método de Euler introduce una espuria explosión de energía. En estas dinámicas, el simulador debe emplear esquemas simplécticos o el clásico **Runge-Kutta de cuarto orden (RK4)**, integrando estados intermedios k_1, k_2, k_3, k_4 que cancelan errores hasta el orden $\mathcal{O}(\Delta t^4)$, asegurando la conservación de los invariantes físicos fundamentales que la academia manda.

2.15. Fundamentación de los Parámetros Físicos Base

Para que Cosmic-Generator produzca un espectáculo visual estable y estéticamente superior antes de que la acústica intervenga, fue imperativo seleccionar un conjunto de parámetros físicos iniciales (base) ubicados en regímenes matemáticos muy específicos. A continuación, el Comité de Física Teórica desglosa la elección de estos escalares codificados en `config.py`.

2.15.1. Motor Gray-Scott (Reacción-Difusión)

El motor principal del simulador arranca frecuentemente con los parámetros:

$$f \text{ (Feed Rate)} = 0,055, \quad k \text{ (Kill Rate)} = 0,062 \quad (2.14)$$

¿Por qué? Estos valores exactos fueron descubiertos por John E. Pearson en 1993 (Complex Patterns in a Simple System”). En el diagrama de fase paramétrico (f, k) del modelo de Gray-Scott, esta coordenada corresponde al régimen de **Patrones de Coral o Laberintos**. Si el *feed rate* fuese más alto, el sistema colapsaría a un estado homogéneo. Si fuese más bajo, la reacción se extinguiría. Este punto crítico permite que, cuando el audio introduzca fluctuaciones vía RMS, el laberinto se fragmente transitoriamente en manchas de Turing (spots) o mitosis celular, retornando al laberinto al cesar la música.

2.15.2. Motor Kuramoto-Sivashinsky (KS)

Los parámetros base dictados para el fuego turbulento son:

$$\nu \text{ (Hiperdifusión)} = 0,1, \quad \text{Advección} = 0,0 \quad (2.15)$$

¿Por qué? La ecuación KS es un modelo arquetípico de caos espaciotemporal. Al fijar el coeficiente de hiperdifusión (∇^4) en 0.1, garantizamos que las inestabilidades de onda larga generadas por la difusión inversa $(-\nabla^2)$ sean amortiguadas exactamente en el rango de los píxeles de nuestra resolución. Esto produce celdas de “fuego” que tienen un diámetro visualmente armónico en resoluciones 720p/1080p, en lugar de ruido blanco de alta frecuencia que resultaría desagradable a la vista.

2.15.3. Motor Cuántico (Gross-Pitaevskii - GPE)

Para la simulación del condensado de Bose-Einstein, el parámetro de interacción g está seteado en:

$$g \text{ (Interacción)} = -2,0 \quad (2.16)$$

¿Por qué? En la física de gases cuánticos diluidos, un valor de $g < 0$ indica interacciones **atractivas** entre bosones. Matemáticamente, esto induce el colapso del condensado y la formación de solitones brillantes (Bright Solitons). Visualmente, esto crea un efecto de "agujero negro luminoso" donde la materia colapsa estéticamente hacia el centro, el cual luego es perturbado y puesto en rotación por los ataques percusivos de la batería.

2.15.4. Motor de Onda Clásica

El amortiguamiento (γ) se establece muy cercano a 0,99 (es decir, decaimiento del 1 %):

$$\gamma = 0,99 \quad (2.17)$$

¿Por qué? Un amortiguamiento nulo ($\gamma = 1,0$) causaría que la energía acústica inyectada fotograma a fotograma se acumule infinitamente, sobreexponiendo la pantalla a blanco (clipping). Un decaimiento del 1 % provee un balance perfecto: las ondas persisten el tiempo suficiente para generar interferencias (patrones de Moiré) al chocar con las paredes periódicas, pero se disipan justo a tiempo para dejar espacio visual al próximo transitorio de la canción.

Parte VIII

Efectos Especiales y Geometría Sagrada (Motores Inéditos)

Capítulo 3

Sistema de Partículas: Los Espíritus Procedurales

Los “Espíritus” son entidades volumétricas reactivas. El sistema de N-cuerpos en `visual_entities.py` simula humo browniano. Las partículas son inyectadas basándose en el parámetro acústico **Kick** (Bombo) y su tiempo de vida es alargado por el contenido armónico.

3.1. Código de Emisión (Extracto)

```
1 # EMITIR NUEVAS PARTICULAS (Humo volumetrico)
2 emission_rate = int(30 + kick * 100) # Mas particulas si hay kick
3
4 # Velocidad: Movimiento organico ascendente/aleatorio
5 base_vx = np.sin(t * 2 + self.noise_offset) * 2.0
6 base_vy = -2.0 - kick * 3.0 # Hacia arriba (gravedad invertida)
```

El humo es renderizado mediante *additive blending* con `cv2.circle`, generando volúmenes suaves en tiempo real.

Capítulo 4

Gestión Lumínica: Bloom y Flash

El motor evita procesadores gráficos complejos aislando altas luces matemáticamente mediante pirámides de downsampling iterativo, creando el efecto **Bloom** o neón.

4.1. Código del Multi-Layer Glow (Extracto)

```
1 def aplicar_bloom(self, img, intensity, threshold=200):
2     # 1. Extraer altas luces
3     gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
4     _, mask = cv2.threshold(gray, threshold, 255, cv2.
5     THRESH_BINARY)
6     bright = cv2.bitwise_and(img, img, mask=mask)
7
8     # 2. Multi-Layer Glow (Piramide Gaussiana)
9     h, w = bright.shape[:2]
10    w1, h1 = max(w // 2, 1), max(h // 2, 1)
11    l1 = cv2.resize(bright, (w1, h1), interpolation=cv2.
12    INTER_LINEAR)
13    b1 = cv2.GaussianBlur(l1, (5, 5), 0)
14    # [...]
```

Capítulo 5

Extracción Lirical y Alineación Forzada

Usando Whisper (`stable-ts`), el código aísla los fonemas en el archivo `motor_lyrics.py`. Para evitar la “Caja Negra” de la IA, usamos *Forced Alignment*, que mapea la letra oficial a marcas de tiempo.

```
1 if external_text:
2     print(" MODO ALINEACION: Usando letra original para
3     sincronizar...")
4     # align() fuerza al modelo a usar este texto y solo buscar
5     los tiempos
6     result = model.align(
7         audio_array,
8         external_text,
9         language='es',
10        progress_callback=self.progress_callback
11    )
```

Esto se envía al renderizador para sobreimprimir texto estilizado al compás exacto de la voz del vocalista.

Capítulo 6

Geometría Sagrada y Caos 3D

Las ilusiones fractales y la simetría sagrada (efecto Caleidoscopio) se implementan invirtiendo y superponiendo tensores, mientras que los atractores de Lorenz (Caos 3D) se proyectan en el plano bidimensional modificando su matriz de rotación con los transitorios del sonido.

```
1 def kaleidoscope(frame, splits=4):  
2     h, w = frame.shape[:2]  
3     # Cortar un fragmento y reflejarlo simetricamente  
4     # para producir una geometria de loto / fractal
```

Esto inyecta espiritualidad geométrica directamente correlacionada con las frecuencias perfectas de las octavas musicales (Tonalidad Pura).